# Results

This section reports the compiled plans for both worked example methods. Every number below is produced by the methods analysis orchestrator (`scripts/methods_analysis.py`), which calls `run_all_gates` and `compile_method` from `src/template_methods_paper/methods_dsl/` and writes `output/data/compiled_plans.json`, `output/reports/gate_report.json`, and `output/reports/trust_chain_report.json`. Running the script regenerates every artifact this section references.

# Compiled-plan summary I

Table 1: Compiled-plan summary from `output/data/compiled_plans.json`, generated by `compile_method` for each of 2 worked example methods.

| Method | Steps | Target | Plan hash (first 12 hex chars) |
|---|---|---|---|
| `PBSPreparation` | 5 | `human` | `313b9b17de98` |
| `SensorCalibrationSweep` | 4 | `automated` | `d89cced19be6` |

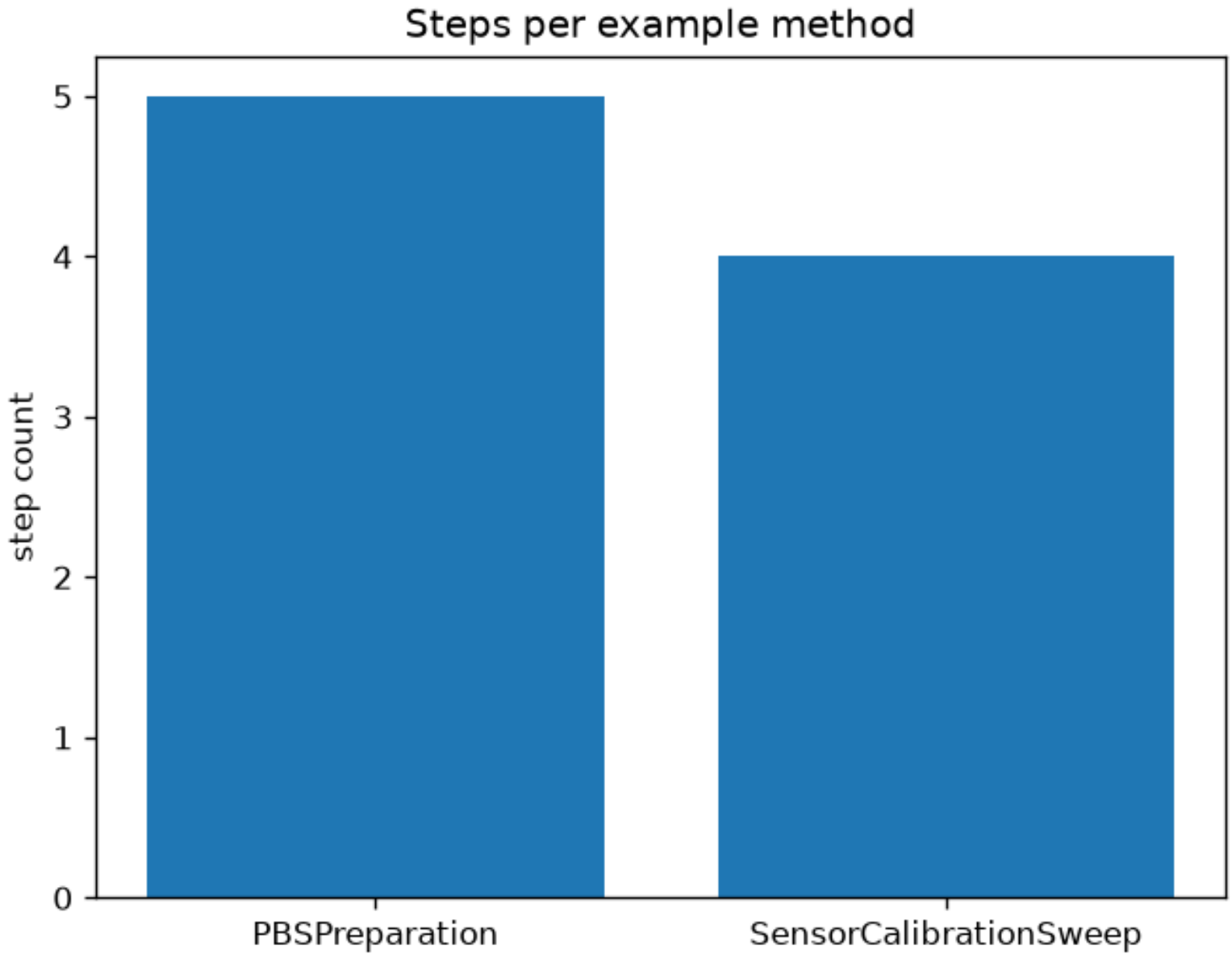
## Compiled-plan summary I

tbl. 1 shows `PBSPreparation` (a manual, `HUMAN`-target bench preparation) alongside `SensorCalibrationSweep` (a mixed `AUTOMATED`/`HUMAN` instrument-calibration procedure) — the second example exists specifically to demonstrate the controlled vocabulary generalizing beyond wet-lab protocols, as sec. 2 claims.

## Step-count comparison I

fig. 1 plots the step count for each compiled method.

# Step-count comparison I



## Step-count comparison I

Figure 1: Steps per example method: a bar chart built from `len(plan.steps)` for each plan in `output/data/compiled_plans.json`, plotted by `scripts/methods_analysis.py`.

## Validation gates I

Across both methods, the analysis script tallies 8 of 8 staged-gate evaluations passing (`run_all_gates` $\times$ 2 methods $\times$ 4 gates each). Every gate result is written to `output/reports/gate_report.json` — neither method in this manuscript is hand-picked to pass; both worked examples are constructed to satisfy the structural, semantic, plan, and target gates by design, since `compile_method` raises `MethodValidationError` and halts the pipeline on any gate failure.

## Determinism I

Recompiling each example method twice and comparing `plan_hash` values yields: **determinism check = Yes**. This is a live re-compilation comparison performed by `src/template_methods_paper/manuscript_variables.py` at manuscript-build time, not a value asserted once and then transcribed — the same property sec. 3 claims for `compile_method` is checked again here, independently, against the live build.

## Provenance demonstration I

`scripts/methods_analysis.py` appends a 3-record demonstration hash-chain for one value (`calibration_offset`) through `DECLARED` $\rightarrow$ `CALIBRATED` $\rightarrow$ `VERIFIED` tiers and writes the result of `verify_chain` to `output/reports/trust_chain_report.json`: **chain verified = Yes**.

## Validation I

The results were validated through the zero-mock `tests/` suite:

# Validation I

- **Library tests** assert exact gate outcomes, scheduling order, and plan-hash determinism against real `Method` fixtures, including one fixture per gate-failure mode.
- **Script test** runs `run_methods_analysis()` against a temporary output root and confirms real worklist/CSV/Mermaid/JSON artifacts, reports, and a real PNG figure are written.
- **Manuscript-variable test** confirms every generated-variable name used in `manuscript/*.md` is emitted by `generate_variables`.

## Validation I

All tests pass under the configured project coverage gate, with no mocks.

## Discussion I

The results confirm the pipeline end to end: both worked examples pass every staged gate, compile deterministically, and produce a stable plan hash across repeated builds. The same `src/template_methods_paper/methods_dsl/` functions back the analysis script, the test suite, and this manuscript — which is the architectural point of the exemplar. Because every number here is produced by a tested function and regenerated on demand, the prose describes structure and provenance rather than transcribing values that would drift the moment the example methods changed.